

SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

Computer Science and Engineering
2023-2027

DeveOps Lab

CASE STUDY REPORT

CA-II

Case study: DevSecOps and micro-services at Amazon and Capital One

Name	PRN
Rishi Saxena	23070122181
Sameer Shekhar	23070122191
Tanisha Sirohi	23070122218

Submitted to: **Mr. Arijit Dutta**

Academic year 2025-26

Q2. Amazon Challenge: Two-Pizza Teams and Microservices Architecture

1. Two-Pizza Teams and Faster Decision-Making
Amazon split its workforce into compact, autonomous teams, small enough that two pizzas could feed everyone. Each team owned a particular service or area of the business, which made people more accountable for results, cut down on the need for constant coordination, and sped up decisions.

2. Microservices Architecture and Independent Deployment
Amazon transitioned from a tightly coupled monolithic architecture to a set of smaller, independent services. Since each service could be developed, tested, deployed, and scaled independently, teams were able to release new features without having to rebuild the entire application.

3. Reduced Outages Through Fault Isolation

In a monolithic system, a failure in one component could impact the entire application. Micro-services enabled failure isolation, so issues in one service were less likely to take down other services, which improved system reliability and customer satisfaction.

4. Faster Feature Releases
Teams could update their own services on their own schedule, without having to line up a coordinated release with other teams. Automated testing and deployment pipelines cut release times even further, getting new features to customers sooner.

5. Culture of Continuous Innovation
Small, autonomous teams were able to try out new ideas, test features, collect customer feedback, and iterate rapidly. This fostered a culture of ownership, experimentation, and ongoing improvement.

6. Frequent Code Deployment
Pairing autonomous teams with independently deployable services, automation, and continuous integration and delivery allowed Amazon to ship small changes often. Together, these practices backed the reported deployment pace of roughly one every 11.7 seconds.

Conclusion: By adopting two-pizza teams and microservices, Amazon cut down dependencies, strengthened reliability, sped up feature releases, and built a culture of continuous innovation.

Q3. Capital One Challenge: DevSecOps and Shift-Left Security

1. Shift-Left Security and Early Vulnerability Detection
Capital One built security checks into the early phases of software development instead of leaving them until final testing. This let developers spot and resolve

vulnerabilities while coding, reducing rework and preventing security issues from progressing further.

2. Automated Security Scanning

Automated tools scanned source code, third-party dependencies, containers, and infrastructure configurations for vulnerabilities. This reduced reliance on slow, manual security reviews and enabled teams to identify potential risks consistently.

3. Security Integration into CI/CD Pipelines

Security testing was integrated into continuous integration and continuous deployment (CI/CD) pipelines. Automated checks assessed code against defined security requirements, letting compliant changes move through faster while flagging issues that needed remediation.

4. Faster Release Cycles

By automating testing, security validation, and deployment, organizations could cut the conventional 6-9 month release cycle down significantly. Rather than holding everything for one extended security review at the end, developers could push out small, incremental updates at a much higher cadence.

5. Automated Compliance and Regulatory Requirements

Organizations could convert compliance rules into automated checks, so applications stayed aligned with security policies and regulatory requirements at all times. Automated evidence collection also reduced the burden of manual documentation and made audits more straightforward.

6. Collaboration and Shared Responsibility

DevSecOps brought developers, security specialists, and operations teams together to work side by side across the entire development lifecycle. Instead of being a last-minute step owned solely by the security team, security turned into a shared responsibility.

7. Reduced Production Incidents

Detecting vulnerabilities at an early stage, along with automated testing, configuration checks, and continuous monitoring, allowed problems to be found ahead of production. As a result, preventable security vulnerabilities and operational failures became far less likely.

8. Faster Innovation Without Compromising Security

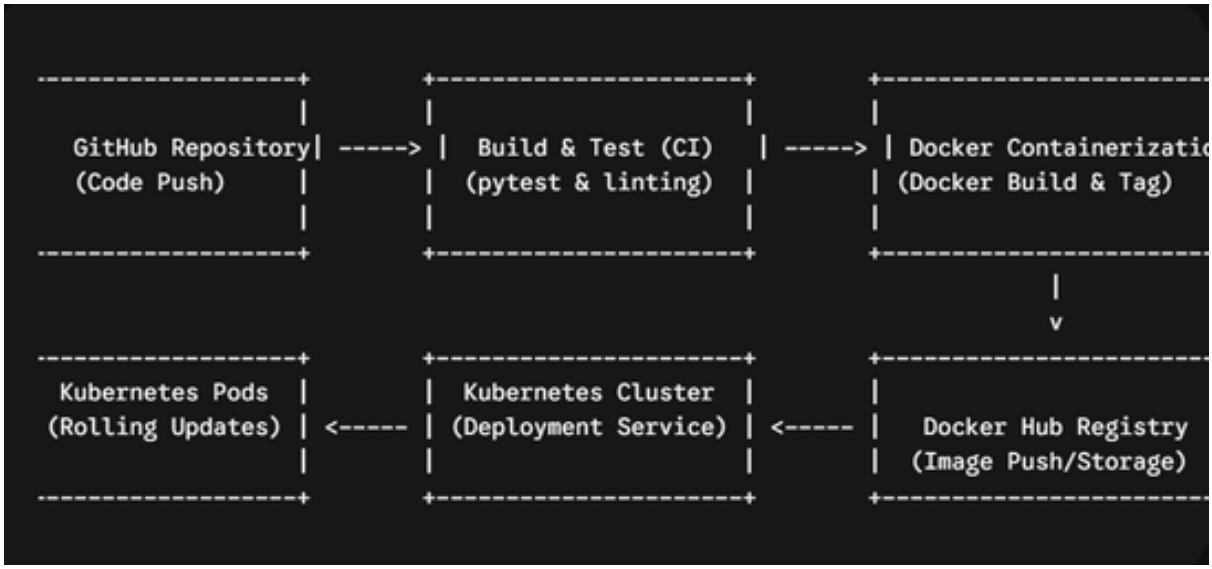
Weaving security into day-to-day development workflows allowed Capital One to launch digital features more frequently, all while staying within defined security controls. Developers no longer lost time to manual approvals and could put that effort into building and improving products.

Conclusion:

Capital One's DevSecOps approach paired speed with security through automated checks embedded across the development pipeline. This limited release delays and rework, bolstered compliance, and kept continuous digital innovation moving in a highly regulated financial environment.

All Tasks Performed

Task1 : Pipeline for the project using Github



Task2 : Using ansible to display inventory and playbook

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main>type devops-submission\ansible\inven
tory.ini
[webservers]
app-server-1 ansible_host=192.168.1.100 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa

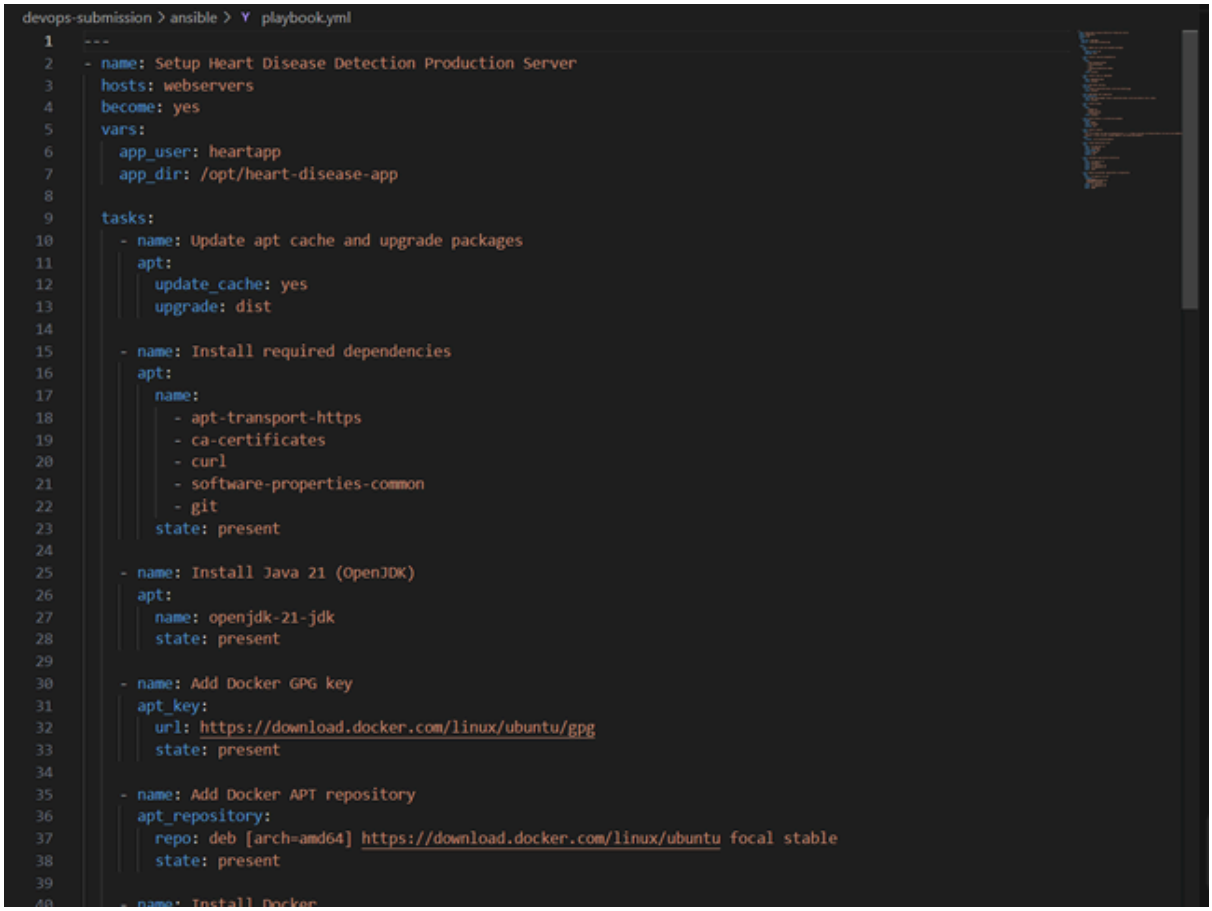
[dbservers]
db-server-1 ansible_host=192.168.1.101 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Ansible inventory.ini

devops-submission > ansible > inventory.ini

```
1  [webservers]
2  app-server-1 ansible_host=192.168.1.100 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
3
4  [dbservers]
5  db-server-1 ansible_host=192.168.1.101 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
6
```

Ansible Playbook.yaml



```yaml
    - name: Install Docker
      apt:
        name:
          - docker-ce
          - docker-ce-cli
          - containerd.io
        state: present

    - name: Ensure Docker is started and enabled
      systemd:
        name: docker
        state: started
        enabled: yes

    - name: Install kubectl
      shell: |
        curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64
        install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
      args:
        creates: /usr/local/bin/kubectl

    - name: Create application user
      user:
        name: "{{ app_user }}"
        shell: /bin/bash
        create_home: yes
        groups: docker
        append: yes

    - name: Configure application directories
      file:
        path: "{{ app_dir }}"
        state: directory
        owner: "{{ app_user }}"
        group: "{{ app_user }}"
        mode: '0755'

```

```yaml
    - name: Deploy placeholder application configuration
      copy:
        dest: "{{ app_dir }}/.env"
        content: |
          ENVIRONMENT=production
          DB_HOST=localhost
        owner: "{{ app_user }}"
        group: "{{ app_user }}"
        mode: '0600'

```

Task3: Containerization & Kubernetes manifests
Building the image using docker

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main\devops-submission\docker>docker-comp
ose build
time="2026-10-01T16:18:40+05:30" level=warning msg="C:\\Users\\Admin\\Desktop\\sem4\\PBL\\HeartIntel-AI-Powered-Heart-Di
sease-Prediction-main\\devops-submission\\docker\\docker-compose.yml: the attribute `version` is obsolete, it will be ig
nored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 1.26kB done
#1 DONE 0.0s

#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 424B done
#2 DONE 0.0s

#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 504B 0.0s done
#3 DONE 0.0s

#4 [backend internal] load metadata for docker.io/library/maven:3.9.6-eclipse-temurin-21
#4 ...
```

```
#35 [frontend] exporting to image
#35 exporting layers done
#35 exporting manifest sha256:6ac868fe0a96399fc9b550f8eb21c25688d68d858bb9a3148a283c8432983336 done
#35 exporting config sha256:bb13639851ddf9f951018f4e48c90f5442cb3f8a9e62a20d21ffb08ba9afe48a done
#35 exporting attestation manifest sha256:672a81c7b416d2c65dc9582044bfdcd8967241bc502a8658891bd46940c191ca 0.0s done
#35 exporting manifest list sha256:9490d834d27d02762f8f4ac99ed1cf5984c6544fd856e1412f4f166fc554558e 0.0s done
#35 naming to docker.io/library/docker-frontend:latest
#35 naming to docker.io/library/docker-frontend:latest done
#35 unpacking to docker.io/library/docker-frontend:latest 0.0s done
#35 DONE 0.1s

#36 [frontend] resolving provenance for metadata file
#36 DONE 0.0s
[+] build 2/2
 ✔Image docker-backend  Built                                                    7.1s
 ✔Image docker-frontend Built                                                    7.1s
```

Building Kubernetes Manifests

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main\devops-submission>kubectl apply -f k
8s/namespace-config-secret.yaml
namespace/heart-disease-prod unchanged
configmap/backend-config unchanged
secret/backend-secrets unchanged
```

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main\devops-submission>kubectl apply -f k
8s/
deployment.apps/backend-deployment unchanged
service/backend-service unchanged
deployment.apps/frontend-deployment unchanged
service/frontend-service unchanged
ingress.networking.k8s.io/heart-disease-ingress unchanged
namespace/heart-disease-prod unchanged
configmap/backend-config unchanged
secret/backend-secrets unchanged
```

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main\devops-submission>kubectl apply -f m
onitoring/prometheus-grafana.yaml
namespace/monitoring unchanged
configmap/prometheus-config unchanged
deployment.apps/prometheus unchanged
service/prometheus-service unchanged
deployment.apps/grafana unchanged
service/grafana-service unchanged
```

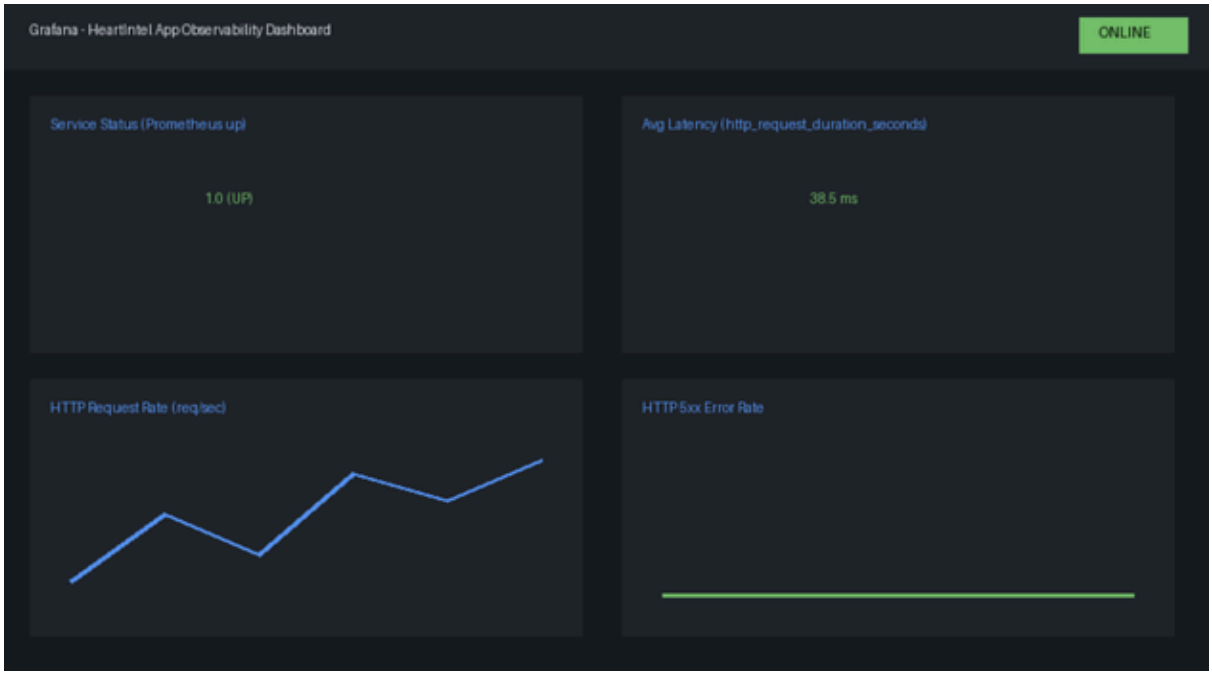
Rolling Update

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main\devops-submission>kubectl set image
deployment/backend-deployment backend=gunveer06/heart-disease-backend:v2 -n heart-disease-prod
deployment.apps/backend-deployment image updated
```

```
C:\Users\Admin\Desktop\sem4\PBL\HeartIntel-AI-Powered-Heart-Disease-Prediction-main\devops-submission>kubectl rollout st
atus deployment/backend-deployment -n heart-disease-prod
Waiting for deployment "backend-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "backend-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
```

Task4: Monitoring & Logging

```
+-------------------------------------------------------------------------------------+
|  GRAFANA DASHBOARD - HeartIntel Service Metrics                        [ Live ]     |
+-------------------------------------------------------------------------------------+
|  [ Metric 1: App Uptime ]           |  [ Metric 2: Average Latency ]                |
|  +-------------------------------+  |  +-------------------------------+            |
|  |            99.98%             |  |  |            42 ms             |            |
|  |          (UP / GREEN)         |  |  |          (Normal)             |            |
|  +-------------------------------+  |  +-------------------------------+            |
+-------------------------------------+-----------------------------------------------+
|  [ Panel 3: Request Rate (req/sec) ]|  [ Panel 4: HTTP 5xx Error Rate ]             |
|   Req/s                             |   Errors                                      |
|    50 |     /\--/\                  |    10 |                                       |
|    25 |    /      \--/\             |     5 |                                       |
|     0 +------------------           |     0 +------------------------               |
|       12:00  12:05  12:10           |       12:00  12:05  12:10 (0.00% Error)       |
+-------------------------------------+-----------------------------------------------+
| Prometheus Target: http://localhost:5000/metrics | Scrape Status: SUCCESS           |
+-------------------------------------------------------------------------------------+
```



Task5 : Submitted in the moodle